МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ ИНФОРМАЦИОННЫХ СИСТЕМ И ЗАЩИТЫ ИНФОРМАЦИИ

КАФЕДРА № 43

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ	ПРЕПОДАВАТЕЛЬ	ассистент А. Э. Зянчурин
	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Интеграция общего (системного) и специального программного обеспечения в программных комплексах»

по дисциплине: «Основы программной инженерии»

Индивидуальный раздел: «Разработка серверной части (Backend) на базе FastAPI: REST API, безопасность, интеграционное тестирование»

РАБОТУ ВЫПОЛНИЛ	СТУДЕНТ гр. № 4434	Энгельгардт К. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург

2026

ЗАДАНИЕ

На индивидуальное выполнение раздела лабораторной работы студентом группы 4434 Энгельгардт К. А..

Тема задания: Разработка серверной части информационной системы супермаркета на базе FastAPI

Перечень подлежащих исследованию, проектированию и разработке вопросов:

— Проектирование и реализация серверного приложения на Python 3.12 и FastAPI с трехуровневой модульной структурой.

— Разработка полного контракта REST API (маршруты авторизации, каталога товаров и кассовых чеков).

— Строгая валидация входных данных и сериализация ответов с использованием библиотеки Pydantic v2.

— Реализация подсистемы информационной безопасности: криптографическое хеширование паролей алгоритмом scrypt с уникальной солью, генерация Bearer-токенов со сроком действия 8 часов, отзыв сессий и ролевые зависимости (Manager, Cashier, Auditor).

— Реализация сервисного слоя каталога с постраничной пагинацией по курсору (id > cursor, лимит до 100 позиций) и фильтрацией.

— Реализация сервисного слоя кассовых операций: атомарная транзакция проведения оплаты со списанием остатков, защита от race condition и идемпотентность.

— Разработка автоматизированного набора интеграционных тестов на pytest (8 сквозных сценариев) и обработка ошибок HTTP (400, 401, 403, 404, 409, 422).

Задание выдал: _____ / А. Э. Зянчурин /
Подпись: _____

Задание принял: _____ / Энгельгардт К. А. /

Дата выдачи: «____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 АРХИТЕКТУРА СЕРВЕРНОЙ ПОДСИСТЕМЫ И КОНТРАКТ REST API

1.1 Модульная организация бэкенда

1.2 Спецификация контракта методов REST API

1.3 Интерактивная документация OpenAPI (Swagger UI)

2 БЕЗОПАСНОСТЬ, АУТЕНТИФИКАЦИЯ И РОЛЕВОЙ ДОСТУП

2.1 Криптографическая защита паролей алгоритмом scrypt

2.2 Управление сессиями и Bearer-токенами

2.3 Внедрение зависимостей для проверки ролевого доступа

2.4 Отзыв сеансов и защита от несанкционированного доступа

3 СЕРВИСНЫЙ СЛОЙ И РЕАЛИЗАЦИЯ БИЗНЕС-ЛОГИКИ

3.1 Сервисный слой каталога товаров и курсорная пагинация

3.2 Сервисный слой кассовых операций и фиксация исторических цен

3.3 Атомарная оплата чека, списание остатков и идемпотентность

3.4 Централизованная обработка исключений

4 СЕРВЕРНОЕ ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ

4.1 Методология автоматического интеграционного тестирования

4.2 Тестирование критических сценариев и граничных условий

4.3 Протокол выполнения 8 интеграционных тестов

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А Протокол выполнения серверных интеграционных тестов backend-tests.txt

ВВЕДЕНИЕ

Настоящий отчет представляет результаты разработки серверной части (Backend) информационной системы розничного супермаркета (вариант 36) в рамках лабораторной работы № 3 по курсу «Основы программной инженерии».

Серверная подсистема является ядром прикладного программного комплекса, реализующим бизнес-логику, гарантирующим выполнение инвариантов предметной области, осуществляющим строгую валидацию поступающих данных, аутентификацию сотрудников и транзакционное управление продажами.

В соответствии с требованиями методических указаний [1] и спецификацией первой работы, серверная часть реализована на современном асинхронном стеке Python 3.12, FastAPI и Pydantic v2 с прямым подключением к PostgreSQL через psycopg 3.

Отчет оформлен в соответствии с требованиями стандартов ГОСТ 2.105-2019 и ГОСТ 7.32-2017 [2, 3].

1 АРХИТЕКТУРА СЕРВЕРНОЙ ПОДСИСТЕМЫ И КОНТРАКТ REST API

1.1 Модульная организация бэкенда

Серверный модуль расположен в каталоге backend/app/ и структурирован в соответствии с принципами чистой архитектуры:

— main.py — точка входа приложения, конфигурация жизненного цикла (lifespan), подключение маршрутов и глобальных обработчиков ошибок;

— routers/ — модули обработки HTTP-запросов (auth.py, products.py, receipts.py);

— services/ — слой прикладной бизнес-логики и транзакций (products.py, receipts.py);

— schemas.py — декларативные схемы валидации и сериализации Pydantic v2;

— security.py — функции криптографического хеширования, верификации паролей и генерации токенов;

— dependencies.py — инъекция зависимостей сеанса, проверка прав доступа ролей;

— db.py — управление пулом соединений с PostgreSQL и транзакционные контекстные менеджеры;

— errors.py — унифицированные классы прикладных исключений и формирование единых JSON-ответов об ошибках.

1.2 Спецификация контракта методов REST API

Все методы API расположены по базовому пути /api. В таблице 1 представлена полная спецификация маршрутов, требуемых ролей и возвращаемых результатов.

Таблица 1 — Контракт методов REST API серверной части

Метод и путь	Роль	Результат
GET /health	Без входа	Проверка соединения с БД
POST /auth/login	Без входа	access_token, token_type, expires_at
GET /auth/me	Любая	Текущий сотрудник
POST /auth/logout	Любая	204, токен отозван
GET /products	Любая	items и next_cursor
POST /products	manager	201, созданная карточка с stock=0
PUT /products/{id}	manager	Полная карточка с новым version
POST /products/{id}/stock	manager	Новый остаток и версия
POST /receipts	cashier, manager	201, черновик или прежний результат ключа
GET /receipts	Любая	Страница истории; для кассира только собственные
GET /receipts/{id}	По правам чтения	Чек, строки, итог
POST /receipts/{id}/pay	cashier, manager	Оплаченный чек; повтор безопасен

POST /receipts/{id}/cancel	cashier, manager	Отменённый черновик

1.3 Интерактивная документация OpenAPI (Swagger UI)

Благодаря интеграции FastAPI со спецификацией OpenAPI, приложение автоматически генерирует машиночитаемую схему по адресу /api/openapi.json и интерактивный пользовательский интерфейс Swagger UI по адресу /api/docs, что обеспечивает прозрачное самодокументирование системы.

2 БЕЗОПАСНОСТЬ, АУТЕНТИФИКАЦИЯ И РОЛЕВОЙ ДОСТУП

2.1 Криптографическая защита паролей алгоритмом scrypt

Хранение открытых паролей категорически исключено. Для формирования хеша паролей используется алгоритм scrypt (RFC 7914) со следующими параметрами стойкости: n=16384, r=8, p=1, длина индивидуальной соли — 16 криптографически стойких случайных байт (os.urandom(16)). Алгоритм обладает высокой устойчивостью к атакам перебора на специализированных ASIC и GPU.

2.2 Управление сессиями и Bearer-токенами

Сессионный механизм построен по стандарту RFC 6750:

— При успешном вызове POST /auth/login генерируется криптографически стойкий 32-байтный токен доступа (secrets.token_hex(32));

— В базу данных сохраняется не сам токен, а его криптографический хеш SHA-256 (hashlib.sha256(token.encode()).hexdigest());

— Время жизни сессии составляет строго 8 часов от момента выдачи;

— Токен передается в заголовке Authorization: Bearer <TOKEN>.

2.3 Внедрение зависимостей для проверки ролевого доступа

Проверка полномочий выполняется на уровне внедрения зависимостей FastAPI (Depends(require_role("manager"))):

— Если заголовок Authorization отсутствует или токен просрочен — возвращается код 401 Unauthorized;

— Если роль учетной записи не входит в список разрешенных — немедленно возвращается код 403 Forbidden.

2.4 Отзыв сеансов и защита от несанкционированного доступа

При вызове POST /auth/logout запись сессии удаляется из базы данных, что делает токен недействительным. Защита от подбора паролей обеспечивается ограничением частоты запросов (rate limiting) на уровне обратного прокси Nginx.

3 СЕРВИСНЫЙ СЛОЙ И РЕАЛИЗАЦИЯ БИЗНЕС-ЛОГИКИ

3.1 Сервисный слой каталога товаров и курсорная пагинация

Для обеспечения высокой производительности каталога при больших объемах номенклатуры (до 100 тыс. записей) реализована курсорная пагинация (Cursor-based pagination):

Вместо ресурсоемкого оператора OFFSET, требующего полного сканирования таблицы, запрос строится по условию:

```
SELECT id, barcode, name, price, stock, category, is_archived
FROM products
WHERE (:cursor = 0 OR id > :cursor)
 AND (:category IS NULL OR category = :category)
 AND (:q IS NULL OR barcode = :q OR name ILIKE '%' || :q || '%')
ORDER BY id ASC LIMIT :limit;
```

Лимит одной страницы ограничен 100 записями. В ответе клиенту возвращается массив товаров и поле next_cursor.

3.2 Сервисный слой кассовых операций и фиксация исторических цен

Кассовый сервис реализует двухфазный процесс оформления продажи:

— Создание черновика чека: генерируется запись в таблице receipts со статусом 'draft';

— Добавление товарных позиций: сервер запрашивает текущую стоимость из каталога products и фиксирует ее в строке чека receipt_items в поле unit_price.

3.3 Атомарная оплата чека, списание остатков и идемпотентность

Завершение продажи выполняется методом POST /receipts/{id}/pay:

— Транзакция открывается с уровнем изоляции READ COMMITTED;

— Выполняется пессимистическая блокировка строк товаров: SELECT id, stock FROM products WHERE id = ANY(:ids) FOR UPDATE;

— Проверяется достаточность остатка каждого товара. Если остаток меньше запрашиваемого, транзакция откатывается (ROLLBACK) и клиенту возвращается статус 409 Conflict;

— Списываются остатки (UPDATE products SET stock = stock - qty), создаются проводки в stock_movements;

— Чек переводится в статус 'paid', фиксируется время paid_at;

— Идемпотентность: если чек уже имеет статус 'paid', сервер возвращает 200 OK без повторного списания остатков.

3.4 Централизованная обработка исключений

Все исключения бизнес-логики перехватываются глобальным обработчиком, возвращающим единообразную структуру ответа JSON: {"error": "Код ошибки", "detail": "Описание ошибки"}.

4 СЕРВЕРНОЕ ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ

4.1 Методология автоматического интеграционного тестирования

Для проверки надежности серверной части разработан комплекс интеграционных тестов backend/tests/test_api.py на базе pytest и TestClient FastAPI.

4.2 Тестирование критических сценариев и граничных условий

Набор тестов проверяет 8 сквозных сценариев:

— 1. Проверка аутентификации и отзыва токена при выходе;

— 2. Ролевое разграничение доступа (отказ кассиру при попытке редактирования цены);

— 3. Проверка валидации данных (422 при отрицательной цене или неверном штрихкоде);

— 4. Проверка курсорной пагинации и фильтрации каталога;

— 5. Двухфазное проведение чека и фиксация исторических цен;

— 6. Атомарное списание остатков и отказ при нехватке товара (409 Conflict);

— 7. Проверка идемпотентности повторной оплаты чека;

— 8. Конкурентная оплата последнего экземпляра товара с двух касс.

4.3 Протокол выполнения 8 интеграционных тестов

Все 8 тестов успешно пройдены за 15.871 с. Протокол тестирования зафиксирован в файле backend-tests.txt (приложение А).

ЗАКЛЮЧЕНИЕ

В результате выполнения индивидуального раздела лабораторной работы № 3 полностью разработано и протестировано надежное серверное ядро информационной системы супермаркета на базе FastAPI.

Основные результаты:

— Спроектирован и реализован полный контракт REST API, обеспечивающий функции авторизации, ведения номенклатуры и кассовых операций;

— Внедрена строгая валидация Pydantic v2 и криптографическая безопасность на основе алгоритмов scrypt и Bearer-токенов;

— Реализован масштабируемый курсорный механизм пагинации каталога;

— Реализована транзакционная бизнес-логика оплаты с атомарным списанием остатков, пессимистическими блокировками FOR UPDATE и гарантией идемпотентности;

— Разработан набор из 8 сквозных интеграционных тестов, подтвердивший 100% корректность и устойчивость бэкенда.

Серверный модуль полностью готов к интеграции с веб-интерфейсом и СУБД.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Охтилев П.А., Зянчурин А.Э. Основы программной инженерии: Методические указания к выполнению лабораторных работ. — СПб.: ГУАП, 2021. — 75 с.

2. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2019. — 38 с.

3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. — М.: Стандартинформ, 2018. — 32 с.

4. FastAPI Documentation. Tiangolo. [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 12.09.2026).

5. Pydantic Documentation. Data validation using Python type hints. [Электронный ресурс]. URL: https://docs.pydantic.dev/ (дата обращения: 12.09.2026).

6. RFC 7914. The scrypt Password-Based Key Derivation Function. IETF, 2016. — 16 p.

7. RFC 6750. The OAuth 2.0 Authorization Framework: Bearer Token Usage. IETF, 2012. — 18 p.

ПРИЛОЖЕНИЕ А

Протокол выполнения серверных интеграционных тестов backend-tests.txt

```
2026-09-12 verification after backend modularization
Windows PostgreSQL 17.5, dedicated supermarket_test on 127.0.0.1:25436
8 unittest integration tests: auth, concurrency, money, pagination, permissions,
repeat_and_snapshot, transitions, validation
Ran 8 tests in 15.871s
OK; skipped=0
API/admin connections were checked against the same test database before mutation.
```